

인터뷰싱크 리스크 관리

도입 결정을 위한 위험 요인과 대응 현황

작성 피하영 · 작성일 2026-09-27 · 인터뷰싱크(면접 일정 자동 조율 도구) · 도입제안서 05·별첨 C 의 근거 문서

본 시스템의 가장 큰 위험은 자동화 실패가 드러나지 않는 상황임. 메일 한 통이 발송되지 않거나 서버가 일시 중단되어도 화면에는 오류가 표시되지 않고, 채용 절차만 멈춤. 따라서 본 시스템의 완성도는 “업무 단계를 얼마나 줄였는가”가 아니라 “실패를 얼마나 빠르게 감지하고 복구할 수 있는가”로 정의함.

요약

대응한 위험

위험	대응 장치	현재 상태
계산 오류로 잘못된 시간 안내	독립 계산 자동 대조, 서로 다른 시간대 4 곳 검증	실제 발생 2 건(회의실 조건 누락·시간대) 수정 완료
미응답 면접관을 “가능”으로 계산	수동 확정 시 확인창·기록, 데이터 점검	계산 방식은 유지, 확정 단계에서 차단
동시 확정에 따른 이중 배정	트랜잭션(일괄 처리)·잠금	일반 확정 차단. 수동 확정은 허용하며 다음 날 점검에서 발견
계산 후 발송 전 상황 변화	발송 직전 데이터 재점검, 이상 시 발송 보류	시스템에 입력되지 않은 변화는 담당자 확인
외부 부품 보안 취약점	취약 부품 교체, 자동 보안 감시	6 건 중 4 건 조치 완료, 2 건 조치 예정(개발 도구)

향후 조치 계획

아래 항목은 조치 완료 전까지 위험이 남아 있음. 각 항목은 도입제안서 04 실행계획의 과제와 연결됨.

위험	조치 전 최악의 경우	조치 계획	시점
로그인·권한 분리 없음	관리 화면 주소를 아는 사람은 누구나 후보자 정보 조회 가능	로그인·권한 분리	도입 전 필수(04 1 단계 A). 완료 전 실제 후보자 정보 입력 금지

개인 메일 계정 발송, 반송 미감지	잘못된 주소도 '발송됨'으로 표시, 개인 계정 정지 시 발송 중단	기업용 발송 서비스 전환, 반송 감지	도입 전 필수(04 1 단계 B, 과제 2-3)
잘못된 확정 메일 발송	발송 후 회수 불가	현재: 담당자 확인 후 발송, 킬 스위치 / 추가: 대상별 발송 중지	도입 전 필수(04 1 단계 B, 과제 4)
킬 스위치 사용 시 기록 오류	중지 기간 미발송 메일이 '발송됨'으로 기록되어 누락	미발송 상태 별도 기록	개발 예정(일정 미정). 그 전에는 재개 후 수동 확인
예약 작업 전체 미실행	매일 점검·재안내·전날 안내가 멈춰도 알아챌 수단 없음	예약 작업 실행 여부 점검	개발 예정(일정 미정). 그 전에는 채용담당자 정기 확인
회의실 정원 미입력	수용 불가 회의실에 면접 배정	회의실 정원 입력	운영 준비 시(04 2 단계, 과제 5)

담당자 업무

도입 후 채용담당자가 직접 확인하거나 처리해야 하는 사항임.

- 수동 확정 시 미응답 면접관 확인창이 표시되면, 해당 면접관의 응답 여부를 확인한 뒤 진행
- 수동 확정은 일정 겹침을 허용하므로, 겹침 여부를 직접 확인
- 확정 메일 발송이 보류되면, 화면의 보류 사유를 확인한 뒤 발송 여부 판단
- 대면→온라인 전환 등 시스템에 입력되지 않은 변경 사항은 발송 전 직접 확인
- 발송 실패로 표시된 건은 대상자 정보 수정 화면에서 주소를 확인한 뒤 재발송
- 전날 안내 메일 발송 실패 시 대상자에게 직접 연락(자동 재시도 없음)
- 회의실 등록 시 정원 입력(미입력 시 인원 제한 없이 계산됨)
- 오류 발송이 의심되면 개발자에게 전체 발송 중지(킬 스위치) 요청

목차

- 요약 — 대응한 위험 · 향후 조치 계획 · 담당자 업무
- 1. 외부 부품(공급망) 리스크 — 약 562 개 외부 부품 사용에 따른 위험과 대응
- 2. 계산 정확성 리스크 — “전원 가능” 판정이 틀리는 6 가지 경로와 방어 장치
- 별첨 A. 프레임워크 선택 근거 / 별첨 B. 백엔드 구조 설계 근거 (개발 검토용)
- 부록 3. Supabase 데이터 접근 보안(RLS) 점검 — 공개 키 접근 시험 결과와 조치

1. 외부 부품(공급망) 리스크

인터뷰링크는 직접 작성한 코드만으로 동작하지 않음. 메일 발송, 화면 구성 등의 기능은 패키지(다른 개발자가 만들어 공개한 기능 부품)를 가져다 씀. 부품에 달려 오는 부품까지 합치면 약 562 개의 외부 부품 위에서 서비스가 동작함. 자체 코드에 문제가 없어도, 이 중 하나에 문제가 생기면 서비스가 중단되거나 공격 경로가 열릴 수 있음.

리스크 정의 — 발생 가능한 문제

리스크	무슨 일인가	영향	발견 당시 상태
가. 알려진 보안 결함이 있는 부품 사용	부품에서 보안 취약점이 발견되어 고친 버전이 이미 나왔는데, 옛 버전을 계속 사용	외부 공격 경로가 열림	확인됨 — 취약점 4 건 미조치 상태(최고 위험 등급 1 건 포함)
나. 부품의 갑작스러운 삭제	제작자가 부품을 삭제하면 새로 설치·배포가 불가능해짐(2016 년 left-pad 사건 — 11 줄짜리 부품 하나 삭제로 전 세계 배포가 멈춤)	우리 코드는 멀쩡해도 배포 중단	낮음(이후 npm 삭제 제한 정책 도입)
다. 예고 없는 대규모 버전 변경	“항상 최신 버전”으로 열어둔 부품은 다음 설치 때 동작 방식이 크게 바뀐 버전이 들어올 수 있음	갑작스러운 오작동	해당 부품 3 개
라. 가짜·방치된 부품 신규 도입	유명 부품 이름을 한 글자 비튼 가짜 부품(타이포스쿼팅)이나, 관리가 끊긴 부품을 설치	악성 코드 유입, 고쳐지지 않는 취약점	새 부품 추가 시마다 확인 필요

‘가’는 가정이 아니라 실제로 확인된 사항임. 점검 결과, 취약점이 있는 버전을 사용 중이었음이 확인됨. 2026-09 점검에서 취약점 6 건이 발견되었으며, next(화면 구성 핵심 부품)·nodemailer(메일 발송 부품) 관련 4 건(최고 위험 등급 1 건 포함)은 조치 완료, 2 건은 조치 예정임.

발견 및 조치 완료 — 4 건

패키지	발견 당시	결과
next	16.2.11 — Critical	☑ 16.3.6 으로 교체, 운영 화면에서도 확인됨
nodemailer	9.0.3 — High	☑ 9.1.1 로 교체
postcss · sharp	next 내부 번들	☑ next 교체와 함께 자동 해결

조치 예정 — 2 건(운영 서비스가 아닌 개발 도구 부품)

패키지	심각도	상태
vitest, @vitest/mocker	보통	자동 수정 요청(PR) 생성됨 — 승인 시 해결
brace-expansion	높음	자동 수정 요청 없음 — 직접 수정 예정

현재 대응과 효과

현재 하고 있는 대응	효과
부품 목록과 정확한 버전을 기록한 파일(package-lock.json)을 저장소에 보관	☑ 효과 있음 — 설치 환경과 무관하게 같은 버전이 재현됨. 단, 해당 버전 자체가 삭제되는 경우는 막지 못함.
CI(코드 변경 시 자동 실행되는 검사)가 내려받은 부품을 임시 보관	△ 부분적 효과 — 부품 저장소(npm)에 문제가 생겨도 당분간 영향 없음. 단, 며칠~1 주일간 사용하지 않으면 자동 삭제되는 임시 방어임.
우리 코드를 검사하는 자동화(린트 · 타입 검사 · 테스트)	✗ 효과 없음 — 자체 코드의 정확성만 확인하며, 외부 부품의 안전성은 확인하지 않음.
부품의 보안 구멍을 자동으로 감시하는 CI 보안 검사 + Dependabot	☑ 신규 도입 — 코드 변경 시마다 자동 확인, 새 취약점 발견 시 교체 요청(PR) 자동 생성. 적용 직후 PR 3 건 자동 생성 확인.

정리 — 버전 고정은 기존에도 적용하고 있었으나, 고정된 버전이 현재도 안전한지 지속 확인하는 절차는 이번 자동 감시 도입 전까지 없었음.

개선 방안 — 비용 대비 효과에 따른 실행 여부

방안	비용	효과	결정
취약한 부품(next · nodemailer) 즉시 업데이트	낮음	높음	☒ 실행 완료
"항상 최신"으로 열어둔 부품 3 개의 버전 고정	낮음	중간	☒ 실행 완료
자동 검사에 부품 보안 검사 단계 추가	낮음(한 번 설정)	높음	☒ 실행 완료 — 이후 계속 자동
새 부품 추가 전 4 단계 확인 습관	거의 없음	중간	☒ 실행(아래 부록 참고)
자체 부품 창고(사설 npm 미러) 운영	높음(서버 추가 운영)	높음	☐ 보류 — 현재 규모엔 과함

- 가. 즉시 조치 — next·nodemailer 를 안전한 버전으로 업데이트, "항상 최신"(latest)으로 설정된 부품 3 개를 특정 버전으로 고정.
- 나. 자동 반복 — CI(코드를 올릴 때마다 자동으로 실행되는 검사)에 부품 보안 검사 단계를 추가해, 취약점이 누적될 때까지 인지하지 못하는 상황의 재발을 방지.
- 다. 운영 원칙 — 새 부품 추가 시 4 단계 확인법(부록)으로 이름·문서·최근 관리 여부·필요성을 확인. 간단한 기능은 부품을 추가하지 않고 직접 작성.
- 라. 업데이트 원칙 — 보안 수정은 자동 보안 검사·Dependabot(보안 수정 자동 제안 도구) 알림 시 가능한 한 빨리 반영, 일반 기능 업데이트는 월 1 회 일괄 반영.
- 마. 보류 — 자체 부품 창고(사설 npm 미러)는 서버를 추가로 운영해야 해 현재 규모에는 과함. 2016 년 left-pad 사건 이후 npm 이 삭제 제한 정책을 도입해 부품이 사라질 가능성은 낮아짐. 회사 규모 확대 시 재검토.

부록. 새 부품 설치 전 4 단계 확인법

AI 나 검색 결과가 추천한 부품도 설치 전에 명령어 한 줄(npm view <부품 이름>)로 대부분 확인할 수 있음. 이 프로젝트의 nodemailer 확인 결과: 이름 nodemailer, 공식 사이트 nodemailer.com, 라이선스 MIT-0, 마지막 수정 2026-09-14(최근까지 관리되고 있음).

확인 항목	확인 방법	위험 신호
① 이름이 맞나	공식 사이트·저장소 주소가 실제	이름이 한 글자 다름, 주소가

	개발자 것인지 확인	영동함(타이포스쿼팅)
② 공식 문서가 있나	homepage-repository 링크를 직접 열어봄	설명서가 비었거나 링크가 끊김
③ 최근에 관리되나	npm view 이름 time.modified	마지막 수정이 2~3 년 전
④ 정말 필요한가	npm view 이름 dependencies	달려오는 부품이 많음, 몇 줄이면 직접 만들 수 있음

정리 — npm view <부품 이름> 명령어 하나로 ①~④를 대부분 확인할 수 있음. 새 부품 설치 전
이 명령을 먼저 실행하는 것을 원칙으로 함.

2. 계산 정확성 리스크

인터뷰링크는 면접관 일정을 모아 “전원이 가능한 시간”을 계산하고, 그 시간을 후보자에게 안내함. 따라서 입력이 모두 정확해도 계산이 틀리면 잘못된 시간이 안내됨. 아래는 실제 코드와 운영 기록에서 확인한 6 가지 경로임.

Q1. “전원 가능”이 잘못 계산되는 경우는?

번호	경우	성격	실제 발생
①	응답하지 않은 면접관을 “가능”으로 계산	설계상 한계	구조적으로 상존
②	계산 로직의 실수(회의실 조건 누락)	계산 실수	발생(2026-08-26)
③	시간대 계산 실수	계산 실수	발생
④	계산 후 발송 전 상황 변화	시점 차이	가능성 있음
⑤	두 사람이 동시에 확정	동시 처리	일반 확정 경로 차단
⑥	회의실 정원 미입력 시 “무제한” 처리	데이터 누락	가능성 있음

① 응답하지 않은 면접관을 “가능”으로 계산

무슨 일인가 — 계산은 “이 시간에 불가능하다고 답했는가”만 확인함. 아직 답하지 않은 면접관은 불가능한 시간이 없으므로, “가능하다고 답함”과 “아직 답하지 않음”이 모두 “가능”으로 계산됨.

근거 — 판정 코드 lib/matching.ts 60 번째 줄 부근. 개발 기록(PROGRESS.md)에도 이 한계가 명시되어 있음.

현재 대응 — 회신 현황 화면(히트맵)에서 “미응답”을 “가능 확인”과 다른 표시(빛금)로 구분함. 수동 확정 시 미응답자가 있으면 확정을 멈추고 확인창을 표시하며, 그래도 진행하면 기록에 남김.

데이터 점검에도 같은 유형(미응답 면접관이 있는 확정)을 추가해 이중으로 확인함(2026-09-27 적용).

② 계산 로직 오류 — 실제 발생

무슨 일인가 — 면접관은 모두 가능하지만 빈 회의실이 없어 실제로는 확정할 수 없는 시간이 “충돌 0(최적 시간)”으로 계산됨. 원인은 회의실 사용 불가 여부를 나타내는 항목(roomBlocked) 하나를 계산에서 누락한 것임.

실제 사례 — 2026-08-26 발견. 면접관 선정과 메일 내용은 모두 정확했으나, 확정할 수 없는 시간이 후보자에게 “가능”으로 안내되고 초대 메일까지 발송됨.

현재 대응 — 독립 검산(같은 규칙으로 별도로 만든 검산용 계산기)과 무작위 대조. 무작위 상황을 두 계산기로 각각 계산해 결과가 다르면 한쪽이 틀린 것으로 판단함. 이 방식으로 ②를 발견·수정했으며, 코드 변경 시마다 자동 실행됨. 검산용 계산기 자체의 오류 위험을 줄이기 위해, 제품 요구사항 문서(PRD)의 확정 규칙 예시로 검산용 계산기를 검증하는 테스트 7 개를 추가함(fast-check 기반, 재현 가능하도록 무작위 시드 고정).

③ 시간대 계산 오류 — 실제 발생

무슨 일인가 — 배포 서버는 UTC(세계 표준시, 한국보다 9 시간 느림)로 동작하는데, 업무시간을 서버 기준 시각으로 계산함. 그 결과 “오전 9 시”로 계산된 시간이 실제로는 한국시간 오후 6 시였음.

추가 위험 — 해외에 있는 후보자의 브라우저에서도 같은 문제가 발생할 수 있음. “전원 가능” 판정이 맞아도 그 시간이 사람마다 다른 시각으로 표시되면, 실제로는 서로 다른 시간에 동의한 것이 됨.

현재 대응 — 서로 다른 시간대 4 곳(한국·UTC·뉴욕·오클랜드) 환경에서 실측 테스트를 실시하며, 4 곳 모두 통과함.

④ 계산 후 발송 전 상황 변화

무슨 일인가 — 계산은 스냅샷(계산 시점의 상태를 저장한 데이터)으로 수행함. 계산 직후부터 확정 메일 발송 전까지 면접관 캘린더에 다른 일정이 생기거나 대면 면접이 온라인으로 바뀌는 등 조건이 달라질 수 있음.

현재 대응 — 발송 직전 데이터 점검(데이터 간 모순 여부 재확인)으로 재검증함. 이상이 있으면 발송을 보류하고 보류 사유를 화면에 표시하며, 담당자가 문제없다고 판단하면 발송할 수 있음. 대면→온라인 전환처럼 시스템에 입력되지 않은 변화는 담당자가 확인해야 함.

⑤ 동시 확정

무슨 일인가 — 두 확정 요청이 거의 동시에 들어오면, 각각 “현재 비어 있음”을 확인한 뒤 둘 다 저장할 수 있음.

현재 대응 — 일반 확정 경로는 트랜잭션(일괄 처리)과 잠금으로 차단함(별첨 B 참고). 실제 데이터베이스에서 동시 확정 시 정확히 1 건만 성공하고, 저장 직후 의도적으로 실패시켜도 전부 이전 상태로 되돌아감을 확인함. 단, 담당자가 직접 확정하는 “강제 확정” 경로는 겹치더라도 의도적으로 통과시키며, 이 경우 겹침은 다음 날 아침 데이터 점검에서 발견됨.

⑥ 회의실 정원 미입력 시 “무제한” 처리

무슨 일인가 — 정원이 입력되지 않은 회의실은 인원 제한을 적용하지 않음(lib/rooms.ts). 4 명이 들어갈 수 없는 회의실이라도 정원이 입력되지 않았다면, “전원 가능·회의실 확보”라는 계산 결과가 실제와 다를 수 있음.

현재 대응 — 정원 미입력을 별도로 처리(isRoomUsable)하고, 관리 화면에 경고를 표시함. 회의실 등록 시 정원 입력이 필요함.

Q2. 발송 전 히트맵 재확인만으로 충분한가?

히트맵 재확인만으로는 부족함. 히트맵도 같은 계산 결과를 표시하므로, ②③⑥ 같은 계산 오류는 히트맵에서도 정상으로 보임. 담당자가 꼼꼼히 확인해도 화면은 잘못 계산된 값을 그대로 표시할 뿐임.

오류 종류	히트맵(사람 확인)으로 잡히나?	실제로 잡는 장치
① 미응답을 “가능”으로 계산	△ 부분적(사람이 놓치지 않아야)	히트맵의 미응답 구분 표시 + 수동 확정 차단 게이트 + 데이터 점검
② 계산 로직 실수	✗ 못 잡음(같은 로직의 결과라서)	독립 검산 무작위 대조
③ 시간대 실수	✗ 못 잡음	여러 시간대에서 실측 테스트
④ 발송 전 상황 변화	△ 부분적(사람이 우연히 알 때만)	발송 직전 재검증(데이터 점검)
⑤ 동시 확정	✗ 못 잡음	트랜잭션 + 잠금
⑥ 정원 미입력	△ 부분적(경고 문구는 있음)	정원 미입력 명시 처리 + 관리 화면 경고

결론 — 히트맵 재확인만은 “담당자가 이미 아는 정보를 시스템이 놓치지 않았는지” 확인하는 장치이지, “계산 자체가 옳은지” 확인하는 장치가 아님. 이에 계산 자체를 검증하는 장치(독립 검산 · 데이터 점검 · 트랜잭션)를 별도로 둬.

그 밖의 구현 사항

- 규칙 기반 설계 — 일정 확정 로직은 후보자 회신 시간 → 면접관 가능 여부 → 회의실 사용 가능 여부의 3 단계 조건을 순차 검증하는 규칙 기반으로 구현함. 창의적 판단보다 추적·재현·수정이 쉬운 것이 중요한 영역이라, 같은 입력에도 결과가 달라질 수 있는 생성형 AI 대신 규칙 기반을 선택함.
- 메일 내용 자동 검사 — 발송 성공 여부뿐 아니라 이름·시간·링크가 본문에 정확히 들어갔는지까지 자동 검사함(sendConfirmationEmail.test.ts).
- 킬 스위치(전체 발송 중지 기능) — 코드 수정 없이 전체 메일 발송을 중지할 수 있음. 실행 주체는 Vercel(서비스 호스팅) 관리 권한 보유자(현재 개발자 1 인)이며, 채용담당자는 개발자에게 요청함. 방법은 설정값(EMAIL_SENDING_ENABLED)을 false 로 변경한 뒤 재배포하는 것으로, 적용까지 수 분이 소요됨. 단, 중지 기간에 건너뛴 메일도 시스템에는 '발송됨'으로 기록되므로, 재개 후 해당 기간의 건을 수동으로 확인해 재발송해야 함.

사후 대응 — 발송 오류 발생 시 감지 → 구분 → 복구

단계	질문	현재 대응	한계
감지	메일 발송이 실패하면 담당자가 알 수 있나?	모든 발송 경로(확정·독촉·전날 안내)에 실패 상태와 사유 표시	전날 안내 메일은 자동 재시도가 없어 담당자가 직접 연락해야 함
감지	서버 중단·배포 실패도 감지할 수 있나?	상태 확인 전용 주소를 만들고 외부 감시 서비스가 주기적으로 확인, 실패 시 이메일 알림	감지·통보까지이며 자동 복구는 없음
감지	확정된 면접들에 모순이 생겼는지도 확인하나?	확정된 건 전체를 매일 한 번 자동 점검(데이터 점검 9 중)	하루 한 번 실행이라 발견까지 최대 하루 소요
구분	“아직 응답 안 함”과 “메일이 애초에 안 감”을 구분할 수 있나?	발송 성공 여부를 별도로 기록해 “응답 대기 중”과 “발송 실패”를 구분 표시	수신자가 실제로 메일을 열었는지는 확인하지 않음
복구	재발송해도 주소 자체가 잘못됐으면 또 실패하지 않나?	실패한 건에서 대상자 정보를 바로 고치는 화면으로 이동하는 경로를 둠	주소가 잘못됐는지는 사람이 판단해야 함 — 반송 메일 자동 감지는 아직 미도입(도입제안서

향후 조치 계획

도입 시 필수(도입제안서 04 실행계획과 연결):

- 개인 메일 계정 → 기업용 메일 발송 서비스 전환(1 단계 B, 과제 2)
- 반송 메일 자동 감지(1 단계 B, 과제 3)
- 로그인·권한 분리(1 단계 A, 과제 1)
- 데이터베이스 유료 요금제 전환(데이터 축적 후 Free → Pro, 도입제안서 03)
- 예약 작업 실행 여부 점검 — 개별 발송 실패가 아니라 예약 작업 전체가 실행되지 않는 상황을 감지(도입 전까지 채용담당자 정기 확인)
- 킬 스위치 사용 시 미발송 기록 — 중지 기간에 건너뛴 메일을 '발송됨'과 구분해 표시(그 전에는 재개 후 수동 확인)

추후 검토(중요도 낮음):

- 회신 링크 위조 방지
- 메일 열람 여부 추적

리스크 관리는 모든 위험을 막는 일이 아니라, 지금 막을 위험과 미룰 위험을 판단하는 일임. 각 판단의 근거를 본 문서에 기록해 향후 재검토와 인수인계 시 활용하도록 함.

별첨 A. 프레임워크 선택 근거

Next.js 는 React 를 대체하는 도구가 아니라, React(화면 구성 도구)에 서버 기능을 더한 프레임워크임. 화면은 React 로 구성하고, 페이지 구성과 서버 API 는 Next.js 가 처리함. 첫 커밋(2026-07-22) 당시 "항상 최신 버전"(latest)으로 설치해 당시 최신인 16 버전이 적용되었고, 이후 16.3.6 으로 고정함. 버전 16 을 별도로 선택한 것은 아님.

표시 기준: ◎ 유리함 · ○ 가능함 · △ 추가 구현이나 학습 필요 · ✕ 상대적으로 불리함

1 단계: 서비스 조건으로 후보 좁히기

인터뷰싱크 구현에 반드시 필요한 조건부터 확인함.

조건	왜 필요한가	Next.js	SvelteKit	Django	NestJS+React
1. 서버 기능	메일 발송, 비밀번호 보호	◎	◎	◎	◎
2. 프론트+백엔드 통합	혼자 개발하므로 관리 대상 최소화	◎	◎	◎	△ 관리 대상 증가
3. 동적인 화면	히트맵, 가능 시간 주기적 재계산	◎	◎	○ 프론트 별도 설계 필요	◎
4. 정기 작업(Cron)	아침 데이터 점검, 독촉 메일	◎	◎	○	○
5. 간편한 배포	과제·시연용 빠른 배포	◎	◎	○	○
6. 여러 DB 작업을 한 번에 처리(트랜잭션)	면접 확정 시 관련 데이터 동시 저장	○	○	◎	○
7. 관리자 화면	면접관·회의실 데이터 관리	△	△	◎ Django Admin	△

걸러진 결과

- 1 번 서버 기능은 네 후보 모두 가능해 판단 기준에서 제외함.
- NestJS+React 후순위 — 프론트엔드와 백엔드를 나눠 관리해야 해 1 인 개발에는 부담이 큼.

- Django 후순위 — 6·7 번은 가장 우수하나, 화면 기능이 많은 서비스라 프론트엔드를 별도로 설계할 부분이 늘어남.
- 남은 후보: Next.js, SvelteKit — 서비스 조건에서 거의 동점임.

2 단계: 개발 여건 조건으로 최종 결정

남은 두 후보는 서비스 조건으로 구분되지 않아, 개발 여건을 기준으로 판단함.

조건	왜 중요했나	Next.js	SvelteKit	Django	NestJS+React
8. AI 활용 자료·예제	AI 와 함께 개발	◎	△	○	○
9. 이전 경험 활용	React+Supabase(Todo 과제) 경험	◎	△	×	○
10. 기존 경험과 이어지는 시작점	초기 세팅 시간 단축	◎ Supabase 공식 Next.js 예제	○	△	○
11. 학습 부담	비개발자에 가까운 상태	○	◎	○	△
12. 개발 중 안정성	버전·설정 문제로 인한 시행착오	△	○	◎	○

결정: Next.js 선택. SvelteKit 은 학습 부담(11)과 안정성(12)에서 유리하나, AI 활용 자료(8)와 이전 경험 활용(9·10)에서 불리함. 기술적으로 가장 우수해서가 아니라, 개발자 1 인의 기존 경험과 학습 부담을 고려할 때 시행착오가 가장 적은 선택으로 판단함. Next.js 의 대표 기능(서버 사이드 렌더링, 검색 노출 유리)은 선택 이유가 아님 — 화면 7 개 중 6 개가 브라우저에서 화면을 그리는 방식("use client")이며, 내부 도구라 검색 노출도 불필요함.

3 단계: 선택에 따른 약점과 대응

Next.js 조합의 약점	실제로 겪은 일	대응
여러 DB 작업을 한 번에 묶기 어려움(6 번)	supabase-js 가 여러 요청을 하나의 트랜잭션으로 묶지 못함(Next.js 탓이 아니라 DB 연결 도구의 한계)	confirm_interview DB 함수를 직접 작성(별첨 B 참고)
관리자 화면 없음(7 번)	회의실을 추가하려면 Supabase 에서 직접 입력해야	회의실 관리 화면 직접 제작

	했음	
설정이 자주 바뀌는 불안정성(12 번)	템플릿의 cacheComponents 옵션 때문에 배포 빌드 실패	옵션 비활성화, 이후 버전 고정(16.3.6)
코드가 서버와 브라우저 중 어디서 도는지 헷갈림(11 번)	한국 시간(로컬)과 UTC(Vercel 서버) 차이로 면접 시간이 9 시간 어긋남	고정 시차 방식으로 수정, 테스트도 시간대를 명시하도록 재작성(2 장 ③ 참고)

별첨 B. 백엔드 구조 설계 근거

요약 — 모든 데이터 접근을 서버 한 곳으로 모아, 향후 로그인 기능을 적용할 지점을 하나로 정함. 면접 확정 시 저장되는 세 곳을 하나로 묶어 “전부 저장 또는 전부 취소”되도록 했으며, 실제 데이터베이스에서 동시 확정 시 1 건만 성공함을 확인함. 회신 현황 화면은 실시간 반영이 아닌 15 초 간격 자동 갱신 방식임. 이하는 개발 검토용 상세 내용임.

데이터베이스는 서버에서 서비스 롤 키(관리자 권한 키)로만 접근

브라우저는 데이터베이스에 직접 접근하지 않도록 구성함. 모든 접근은 서버(API 라우트)를 거치며, API 라우트는 RLS(행 단위 접근 제어)를 우회하는 서비스 롤 키로만 Supabase(데이터베이스)에 접근함.

- 이유 — 현재 로그인·권한 분리 기능이 없어, 관리 화면 주소를 아는 사람은 누구나 서버를 통해 데이터를 조회할 수 있음. 여기에 브라우저가 Supabase 익명 키로 직접 접근하는 구조까지 더하면, RLS 정책이 없는 현 상태에서 접근 통로가 하나 더 생김. 실제 후보자 정보는 로그인·권한 분리(도입제안서 04 1 단계 A) 완료 전까지 입력하지 않음.
- 모든 접근을 서버 코드(/api/*) 한 곳으로 모아두면, 향후 로그인·권한 분리를 적용할 지점이 하나로 정해짐 — 여러 화면에 흩어진 접근 경로를 각각 찾아 수정할 필요가 없음.

확정 시 저장 3 곳을 트랜잭션(일괄 처리)으로 묶은 이유

면접 확정 1 건은 면접 정보, 면접관별 사용 중 시간(busy_slots), 회의실 사용 중 시간(busy_slots) 세 곳에 나뉘어 저장됨. 데이터베이스 연결 도구(supabase-js)는 여러 요청을 하나의 트랜잭션으로 묶지 못함(요청 1 건이 트랜잭션 1 건). 이에 따라 두 가지 위험이 있었음.

1. 반쪽 확정 — 면접 정보에는 “확정”으로 기록되었으나 캘린더는 비어 있는 상태
2. 동시 확정 — 두 요청이 각각 “비어 있음”을 확인한 뒤 둘 다 저장하는 상태

세 번의 저장을 confirm_interview 라는 DB 함수 안으로 옮기고 한 번만 호출(rpc)함. 데이터베이스 API(PostgREST)가 요청 1 건을 트랜잭션 1 건으로 처리하므로, 세 곳이 전부 저장되거나 전부 취소됨.

- 잠금은 두 겹 — 확정 작업 전체에 잠금 하나(pg_advisory_xact_lock)를 걸어 확정 작업끼리 동시에 실행되지 않도록 함(잠금이 하나뿐이라 서로 기다리다 멈추는 교착 상태가 발생하지 않음). 그 위에 실제로 저장할 면접관·회의실 행을 잠금(select ... for update).

- 잠금 뒤 재확인 — 호출한 쪽이 판단할 때 읽은 데이터는 그사이 바뀌었을 수 있어, 이 재확인이 이중 배정을 실제로 막는 지점임. 겹치면 오류 코드 PT409 로 거부함.
- 예외 처리 블록(EXCEPTION)을 두지 않음 — 예외를 처리하면 중간 저장 지점(세이브포인트)까지만 되돌아가, 오류를 다시 발생시키지 않는 한 함수 전체가 취소되지 않음. 이에 예외가 그대로 전달되도록 함.
- 수동 확정은 겹침을 허용(force) — 수동 확정은 담당자 판단을 우선해 일정 겹침을 허용하는 설계이며, 겹침 여부는 담당자가 확인해야 함. 트랜잭션 도입 후에도 이 설계는 유지하며, 여기서 얻는 효과는 "겹침 차단"이 아니라 "일부만 저장되는 상황 방지"임.
- 판단 로직은 옮기지 않음 — 가능한 시간을 판단하는 로직(lib/matching.ts)은 TypeScript 에 남겨 자동 테스트(vitest)로 검증함. DB 함수 언어(plpgsql)는 타입 검사(tsc)와 자동 테스트가 적용되지 않아, 판단 로직을 옮기면 검증 수단이 사라짐. DB 함수는 "잠금, 재확인, 세 곳 저장"만 수행함.

데이터 점검은 유지 — 트랜잭션은 "작성한 로직대로 전부 저장되거나 전부 취소됨"을 보장할 뿐, "로직이 옳은지"는 보장하지 않음. 함수 자체의 오류, 사람이 직접 수정한 데이터, 정책상 허용한 예외(회의실 없는 수동 확정)는 여전히 사후 데이터 점검으로만 발견됨.

실시간 반영(Realtime) 대신 15 초 간격 자동 갱신을 선택한 이유

회신 현황 화면은 15 초마다 자동 갱신됨(실시간 반영 아님). 실시간 반영 기능(Supabase Realtime)은 테이블별 활성화와 접근 제어(RLS) 설정이 필요하고, 구독 직후 변경 사항이 누락되거나 화면 이동 후 이전 연결이 남는 문제가 알려져 있음. 이 화면은 담당자만 보고 응답이 분 단위로 들어오므로, 실패해도 다음 주기에 자동 복구되는 주기적 갱신(폴링)이 위험 대비 이득이 크다고 판단함. 갱신 경로를 하나로 두어, 향후 방식을 바꿀 때도 해당 지점만 수정하면 됨.

부록 3. Supabase 데이터 접근 보안(RLS) 점검

요약 — 데이터베이스 자체의 접근 제어(RLS)는 적용되어 있으며, 공개 키로 접근하면 모든 테이블이 차단됨을 실제 조회로 확인함. 점검 중 읽기용 뷰 1 개가 RLS 를 우회하는 것을 발견해 조치를 완료함(2026-09-27). 다만 로그인·권한 분리가 없어, 서버를 거치면 누구나 데이터에 접근할 수 있는 위험은 남아 있음.

왜 점검했나

Supabase 는 브라우저가 공개 키(웹페이지에 포함되어 누구나 볼 수 있는 접속 키)로 데이터베이스에 직접 접근할 수 있는 구조임. 따라서 RLS(행 단위 접근 제어)를 켜고 규칙을 정해야만 안전하며, 켜지 않으면 공개 키만으로 누구나 데이터를 읽고 수정·삭제할 수 있음. AI 코딩 도구로 만든 앱에서 RLS 가 꺼진 채 배포되어 데이터가 통째로 노출된 사고(CVE-2025-48757)가 실제로 있었음.

판단 기준: 공개 키 + RLS 켜짐 = 안전 · 공개 키 + RLS 꺼짐 = 누구나 접근 가능한 데이터베이스 (출처: Supabase 공식 문서, CVE-2025-48757)

점검 결과

점검 항목	결과	확인 방법
테이블 4 개(면접관·회의실·면접·회신 요청) RLS 적용	☒ 4 개 모두 켜짐	설정 파일(schema.sql) 확인
공개 키로 실제 접근	☒ 4 개 모두 0 건 조회(실제 저장 건수 2·9·3·13 건)	공개 키로 직접 조회 시험(읽기만 수행)
접근 규칙(정책)	☒ 규칙 미설정 = 공개 키 접근 전면 차단	브라우저가 데이터베이스에 직접 접근하지 않는 구조(별첨 B)에 맞춘 설정
서비스 롤 키 노출 여부	☒ 서버 코드에서만 사용	코드 전체 검색
확정 처리 DB 함수(confirm_interview)	☒ 공개 키 실행 권한 회수	설정 파일 확인
읽기용 뷰(회의실 요약)	⚠ 발견 당시 공개 키로 3 건 조회 → ☒ 조치 완료	조치 전후 공개 키 조회 비교
로그인·권한 분리	☒ 없음 — 남아 위험	운영 주소에 비로그인 상태로 접속해 데이터가 조회됨을 확인

발견 및 조치 — 읽기용 뷰의 RLS 우회

뷰(여러 테이블을 보기 좋게 모아 둔 요약표)는 기본 설정상 만든 사람(관리자) 권한으로 실행되어 RLS 를 적용받지 않음. 담당자가 관리 화면(Table Editor)에서 훑어보기 위해 만든 회의실 요약 뷰가 공개 키로 조회되었음. 이처럼 단순한 뷰는 수정·삭제 경로가 될 가능성도 있었음(실제 데이터를 지워야 확인되는 시험이라 수행하지 않음).

- 조치 1 — 뷰를 조회하는 사람의 권한으로 실행하도록 변경(security_invoker). 공개 키로 조회하면 테이블의 RLS 가 그대로 적용됨.
- 조치 2 — 공개 키의 뷰 접근 권한 회수. 관리 화면 전용 뷰라 외부에 열어둘 이유가 없음.
- 조치 3 — 뷰를 만드는 기존 설정 파일 5 곳에도 같은 설정을 넣어, 설정 파일을 다시 실행해도 보호 없는 뷰로 돌아가지 않게 함.

대상	조치 전	조치 후
회의실 요약 뷰 — 공개 키	3 건 조회됨	접근 거부(permission denied)
회의실 요약 뷰 — 서비스 롤 키	3 건	3 건(변화 없음)
테이블 4 개 — 공개 키	0 건	0 건(변화 없음)
서비스 동작(회의실 조회)	정상	정상(앱은 이 뷰를 사용하지 않음)

근거: [supabase/migration_view_security_invoker.sql](#) · [GitHub PR #12](#)

남은 위험 — 로그인·권한 분리 없음

RLS 는 데이터베이스의 정문(공개 키)을 지키는 장치임. 인터뷰싱크는 모든 접근이 서버(API 라우트)를 거치고, 서버는 RLS 를 우회하는 서비스 롤 키를 사용함. 그런데 서버 앞에 로그인 확인이 없어, 주소를 아는 사람은 누구나 서버를 통해 후보자 정보를 조회하고 면접 건을 삭제할 수 있음. 결과적으로 “RLS 가 꺼진 데이터베이스”와 같은 효과임.

현재는 가상 데이터만 사용하는 시연용이라 실제 피해는 없으나, 실제 채용 데이터를 입력하기 전에 로그인·권한 분리가 반드시 필요함(요약 “향후 조치 계획” · 도입제안서 04 실행계획 1 단계 A, 과제 1).

운영 원칙 — 테이블·뷰를 새로 만들 때

- 테이블은 만들 때 RLS 를 켜. 공개 키로 열어줄 필요가 없으면 규칙 없이 두어 접근을 전면 차단함.
- 뷰는 security_invoker 를 켜고, 관리 화면 전용이면 공개 키 권한을 회수함.

- DB 함수는 공개 키 실행 권한을 회수함.
- 만든 뒤 공개 키로 직접 조회해 0 건 또는 접근 거부가 나오는지 확인함. AI 코딩 도구에 "RLS 켜고 정책 넣어줘"라고 요청한 경우에도 결과를 직접 확인함.
- Supabase 관리 화면의 보안 점검 기능(Security Advisor)으로 RLS 누락·뷰 권한 문제를 정기적으로 확인함.

정리 — 데이터베이스의 정문(RLS)은 잠겨 있고, 요약표(뷰)를 통한 우회 경로도 막았음. 남은 것은 서버 창구의 신분 확인(로그인)이며, 실제 데이터 투입 전 필수 과제임.